

Processus MLOps Complet : De l'Expérimentation à la Production

Ce document présente les étapes pas à pas du cycle de vie d'un projet de Machine Learning industrialisé (MLOps), structuré en 5 grandes phases.

1. Exploration & Développement (Le bac à sable)

- **Versionnage initial (Git)** : Initialisation du dépôt dès le premier jour. Tout code doit être traqué.
- **Analyse exploratoire (EDA)** : Compréhension des données, identification des anomalies et des corrélations.
- **Prototype en Notebook (Jupyter)** : Expérimentation rapide pour valider la faisabilité technique.
- **Modèle Baseline** : Entraînement d'un modèle ultra-simple (ex: régression linéaire, arbre de décision basique) pour fixer une métrique de performance de référence.

2. Industrialisation & Optimisation (Le passage au propre)

- **Refactoring (.py)** : Nettoyage et structuration du code des notebooks en scripts Python modulaires, réutilisables et testables.
- **Préparation des données** : Création d'un pipeline robuste de chargement et de preprocessing des données.
- **Entraînement de plusieurs modèles** : Évaluation de différentes architectures d'algorithmes.
- **Optimisation des hyperparamètres** : Recherche fine de la meilleure configuration avec des outils comme **GridSearch** ou **Optuna**.
- **Tracking & Registry (MLflow)** : Enregistrement systématique de tous les essais (paramètres, métriques, courbes) et centralisation du meilleur modèle dans le *Model Registry*.

3. Packaging & CI/CD (Prêt pour la prod)

- **Création de l'API (FastAPI)** : Développement de l'infrastructure de service pour exposer le modèle via des points de terminaison (endpoints) d'inférence.

- **Création du Frontend (Streamlit)** : Développement d'une interface utilisateur intuitive pour faciliter la manipulation et la consommation du modèle par les équipes métiers.
- **Dockerisation** : Création d'images Docker pour isoler l'API et le Frontend, garantissant un comportement identique entre le local et la production.
- **CI/CD (GitHub Actions)** : Automatisation de l'exécution des tests unitaires, du build des images Docker et de leur push vers un registre de conteneurs à chaque commit.

4. Déploiement & Orchestration (La mise en ligne)

- **Déploiement (Infrastructure)** : Déploiement des conteneurs sur un serveur virtuel (VPS) via Docker Compose ou sur un environnement Cloud managé (AWS, GCP).
- **Orchestration (Airflow)** : Planification (scheduling) et automatisation des tâches récurrentes :
 - Récupération et préparation des nouvelles données.
 - Exécution des prédictions par lot (Batch prediction).
 - Déclenchement automatique du réentraînement du modèle.

5. Monitoring & Maintenance (Le chaînon manquant)

- **Suivi des performances en production** : Surveillance de la précision du modèle face aux données réelles du marché.
- **Détection du Data Drift** : Analyse de la dérive des données d'entrée par rapport aux données d'entraînement initiales (avec des outils comme *EvidentlyAI*).
- **Supervision de l'infrastructure** : Collecte des métriques système (CPU, RAM, latence de l'API) avec *Prometheus* et visualisation sur des dashboards *Grafana*.
- **Boucle de feedback** : Alerte en cas de dégradation des performances pour réinjecter les données fraîches dans la phase 1 ou 4 pour réentraînement.